# Ders 5 Çalışma Özeti

## Ders 5 Çalışma Özeti

### Genel Konular

- Proje grupları ve proje yazılımı seçimi hakkında genel bilgilendirme.
- Test konusunun tekrarı: V model, doğrulama (verification) ve geçerleme (validation), test süreci adımları, test senaryosu (test case), test tipleri, hata düzeyleri.
- Sistem testinin kalite özellikleriyle ilişkisi ve McCall kalite modelinin hatırlatılması.
- Yazılım ölçümü (software metrics): neden ölçülür, yöneticiler ve teknik kişiler açısından ölçme ihtiyacı.
- Halstead metriği, McCabe döngüsel karmaşıklık (cyclomatic complexity) ölçüsü.
- Büyüklük ölçütleri: satır sayısı (LOC, KLOC) ve işlev noktası (function point).
- Yazılım güvenirliği (reliability), güvenirlik modelleri ve güvenirliği sağlama adımları.
- Nesneye yönelik yazılımlar için CK (Chidamber-Kemerer) metrikleri.
- Proje gereksinim belgesi (SRS) için IEEE 830 formatı ve bir öğrenci projesi örneği.

### Hocanın Özellikle Vurguladığı Kısımlar

- Proje için seçilen yazılımın kaynak kodu elinizde olmalıdır, çünkü birim testi istenecektir.
- Seçtiğiniz kodun diline uygun bir birim testi aracı bulabileceğinizden emin olun. Örneğin Java için JUnit kolay bulunur.
- Grup listesi kesindir. Ders sonunda projeler bu listeye göre değerlendirilecektir; sonradan kişi eklenmez.
- Sistem testi, fonksiyonel olmayan kalite özellikleriyle (güvenlik, dayanıklılık, performans) eşleşir.
- Tom DeMarco: “Ölçemediğinizi kontrol edemezsiniz.”
- McCabe değeri bir modülde 10’u geçmemelidir. Geçerse modül alt modüllere bölünmelidir; aksi halde test etmek zorlaşır.
- Bir yazılımı tümüyle test etmek mümkün değildir. Amaç mümkün olduğunca çok hatayı yakalamaktır.
- Güvenirlik için yalnızca bir model uygulamak yetmez; süreci de iyi yönetmek gerekir.
- SRS belgesinde esas istenen 4. ve 5. bölümlerdir (fonksiyonel ve fonksiyonel olmayan gereksinimler). İlk üç bölüm kısa olabilir.

### Kısa Tekrar Notları

- Doğrulama (verification): ürünü doğru yolla oluşturduk mu? Birim ve bütünlük testinde ağırlıklıdır.
- Geçerleme (validation): doğru ürünü oluşturduk mu? Müşteri gereksinimleri yazılımda var mı?
- Test senaryosu iki düzeyde düşünülür:
  - Kullanıcı testinde: özel bir gereksinim senaryosu (örneğin çift anadal öğrencisinin ders kaydı).
  - Birim testinde: en alt modülün hangi girdide hangi çıktıyı vermesi gerektiği; algoritmadaki bütün yolları deneyecek test verisi.
- Bileşen testi: bileşen (component) mimarisi kullanılmışsa bileşenlerin test edilmesidir.
- Kabul testi = onaylama testi (yazılım, gereksinimler, kullanıcı belgeleri) + konfigürasyon (düzen) incelemesi.

- Halstead metriği: işleç (operator) ve işlenen (operand) sayılarından program uzunluğu, hacmi ve gücünü hesaplar.
- McCabe ölçüsü: akış grafındaki bağımsız yol sayısını verir.
- Doğrudan büyüklük ölçümü: satır sayısı (LOC, KLOC). Dolaylı ölçüm: işlev noktası (function point).
- Güvenirlik: geçmiş bilgilere ve teste dayalı başarı oranıdır. Başarılı süre / toplam işletim süresi.
- CK metrikleri: WMC, DIT, NOC, CBO, RFC, LCOM.

## Detaylı Açıklamalar

### Proje Duyuruları

- Hoca grup listelerinin çoğunu almıştır. Listeyi kayıt altına alıp sonra sınıfla paylaşacaktır.
- Konusu reddedilen grup yoktur.
- Uygun yazılım örnekleri: web uygulaması, yönetim bilgi sistemi uygulaması, daha teknik bir yazılım. Başka derslerin (Nesneye Dayalı Sistemler, Yazılım Mühendisliği) proje ödevleri de kullanılabilir.

### V Model ve Doğrulama/Geçerleme Tekrarı

- Kodlamadan hemen sonra birim testi yapılır: “Algoritma doğru yolla oluşturulmuş mu?” Bu doğrulamadır.
- Bütünlük testinde gereksinim analizindeki işlemlerin tasarıma doğru aktarılıp aktarılmadığına belgelerden izlenerek ve çalıştırılarak bakılır. Bu da doğrulamadır.
- Geçerlemede müşterinin istediği işlevlerin ve ürün spesifikasyonunun sistemde sağlanıp sağlanmadığına bakılır.
- Test adımları geliştirme belgelerini kullanır:
  - Bütünlük testi: tasarım belgesi.
  - Sistem testi: gereksinim belgesi.
  - İşlemsel kullanım: müşteriden alınan bilgiler ve ürün istek bildirimleri.

### Test Senaryosu Örneği

- Öğrenci bilgi sistemi örneği:
  - Çift anadal yapan öğrencilerin durumunu sorgulama veya ders kaydı yapma bir test senaryosudur.
  - Mühendislik tamamlama öğrencilerinin ders kaydı başka bir senaryodur.
  - Her kullanıcı grubu için ayrı senaryo oluşturup sistemin doğru çalıştığı test edilir.
- Birim testinde test senaryosu daha mikro düzeydedir. Her modülün kendine düşen işi yaptığı, saydam kutu biçiminde iç yapıya bakılarak test edilir.

### Bütünlük, Regresyon ve Sistem Testi Tekrarı

- Tek tek hatasız modüller birleşince hata toleransları büyüyebilir, ortak veri yapıları sorun çıkarabilir, birleşik yapı istenen işi yapmayabilir.
- Bütün yazılımı birleştirip tek seferde test etmek verimli değildir. Artırmalı sınama daha iyidir.
- Yukarıdan aşağı: alttaki modüller önce kukla modül (stub) olarak düşünülür; aşağı inildikçe gerçek modüller eklenir.
- Aşağıdan yukarı: önce en alttaki modüller tamamlanır. Örneğin Module 7’nin çağırdığı Module 1 yerine konur; Module 8 ve Module 9 için de aynısı yapılır. Bu kollar paralel ilerleyebilir.
- Regresyon testinde daha önce test edilmiş bölümler için test araçlarından yararlanılabilir.
- Sistem testi, yazılımın sistem donanımıyla uyumlu çalıştığını onaylar.
  - Güvenlik testi: zararlı dış müdahalelere karşı koruma.
  - Dayanıklılık (stres) testi: aşırı yükte dayanıklılık.
  - Yetenek (performans) testi: işlem süresi ve yetenek baştaki isteklere uygun mu? Cevap süresi, geri dönüş süresi gibi değerler kontrol edilir.

- Performans testi şu sorulara cevap verir: Sistem gereksinimleri karşılanıyor mu? Normal şartlarda sistem nasıl davranıyor? Trafik artınca işlem süresi ve işlevsellik nasıl etkileniyor? Kullanıcı seviyeleri arasında performans sorunu var mı?

**Sistem Testi ve Kalite Özellikleri**

- Hoca McCall'ın kalite özelliklerini hatırlattı: kullanıcı, geliştirici ve işletici olmak üzere üç bakış açısı.
- Kullanıcı yazılıma dışarıdan bakar. Yazılımın iyi test edilip edilmediğini fonksiyonel olmayan özelliklerden anlar: güvenlik, dayanıklılık, verimlilik.
- Bir öğrencinin cevabı: dayanıklılık, kullanıcı sayısı belirlenen sınırın üzerine çıktığında veya sınır değerlerde sistemin aynı verimle çalışıp çalışmadığıdır.
- Sonuç: sistem testi, fonksiyonel olmayan kalite özelliklerinin test edilmesidir.

**Onaylama ve Kabul Testi Tekrarı**

- Onaylama testinde kullanıcıya benzer bir grup veya doğrudan kullanıcı yazılımı çalıştırır.
- Gereksinimlerden yola çıkılır: bütün gereksinimler yazılımda sağlanıyor mu?
- Kullanıcı belgeleri (yardım mekanizması, kâğıt açıklamalar) izlenerek bütün gereksinimlere ulaşılabiliyor mu, buna bakılır.
- Konfigürasyon (düzen) incelemesi:
  - Kullanıcı belgeleri, tasarım belgeleri, kayıtlar ve test belgeleri istenen formatta mı?
  - Birbiriyle ilişkili mi? Örneğin hangi program hangi test verisiyle denendi, hangi tasarım belgesi hangi programı anlatıyor?
- Alfa testi: yazılım ekibinde kullanıcıya benzer bir grup kontrol eder. Beta testi: uygulama potansiyel kullanıcılara verilir.

**Test Tipleri Tekrarı**

- Saydam kutu testi: işlemsel tasarımın kontrol yapısı test edilir. Algoritmadaki bütün yolları deneyecek test verisi üretilir; döngülerin başlangıç, bitiş ve artış değerleri kontrol edilir; iç veri yapıları kontrol edilir.
- Temel yol testi adımları: kaynak koddan akış grafı çizilir, McCabe ölçüsüyle temel yol sayısı bulunur, her yol için giriş veri grubu hazırlanır, test uygulanıp beklenen sonuçla karşılaştırılır.
- Döngü testi: başlama hatası, sonsuz döngü ihtimali ve sınır hatası aranır.
- Kara kutu testi: işlevsel gereksinimler denenir. Bulunan hatalar: hatalı ve eksik işlemler, veri yapısı ve veritabanı erişim hataları, performans hataları.
  - Eş değerli bölümleme: veri uzayı alt sınıflara bölünür, her sınıftan bir örnekle uygulama çalıştırılır. Geçersiz girdi de verilir ve yanıtı gözlenir.
  - Sınır değer analizi: hatalar merkezden çok kenarlarda toplanır. Örneğin kullanıcı sayısının maksimum seviyesine yakın bir değerle çalıştırıp programın çöküp çökmediğine bakılır.
- Hata düzeyleri: ölümcül ve büyük hatalarda genellikle teste devam edilmez. Bazı ekipler büyük hatada teste devam eder ama yazılımı teslim etmez. Sınıflama her kurumun kendi yapısına göre belirlenir; altı grup zorunlu değildir.

**Yazılım Ölçümü Neden Yapılır?**

- Tom DeMarco: ölçme yapılamazsa kontrol edilemez.
- Yöneticiler ölçme ister, çünkü:
  - Yeni yazılımın maliyetine karar vermek için bitmiş yazılımları ölçerler.
  - Çalışanların ne kadar iş yaptığını, iş gücünü ve ücretlerini belirlemek için personel üretkenliğini ölçerler.
  - Farklı projelerde geliştirilen ürünlerin kalitesini ve büyüklüğünü kıyaslamak isterler.
  - Projeler için ölçülebilir hedefler koymak isterler (örneğin sistemin ne kadar güvenilir olduğu).
  - Kullanılacak yöntemlerin ve geliştirme araçlarının etkinliğini saptamak isterler.
- Teknik kişiler ölçme ister, çünkü:
  - Bitirilen ürünün kalite ve performans gereksinimlerini daha kolay belirlemek isterler.

  - Sistemin kalitesini daha kolay izlemek isterler.
  - Tasarım değişikliklerini ve gözden geçirme/test adımlarında bulunan hataları değerlendirmek için süreç ölçümleri yaparlar.
  - Var olan ürünlerden öğrenip sonraki ürünlerde yöntem seçmek isterler. Örneğin nesneye yönelik yazılımlarda bağımlılık (coupling) ve uyum (cohesion) ne ölçüde olmalı, buna karar verirler.

**Halstead Metriği**

- Kaynak programın aritmetiksel (hesaplama) karmaşıklığını ölçer. Eskiden beri kullanılır.
- Sayılan değerler (slayttaki gösterimle):
  - $m$: programdaki farklı işleç (operator) sayısı.
  - $k$: programdaki farklı işlenen (operand) sayısı.
  - $M$: işleç yinelenmelerinin toplamı.
  - $K$: işlenen yinelenmelerinin toplamı.
- Hesaplanan ölçütler:
  - Program uzunluğu: $U = m \log_2 m + k \log_2 k$
  - Program hacmi: $H = (M + K) \log_2(m + k)$
  - Program gücü (effort): $G = H/O$. Burada $O$ oransal hacimdir ve daima 1'den küçük çıkmalıdır.

**McCabe Döngüsel Karmaşıklık Ölçüsü**

- Programın kontrol akışına dayanarak karmaşıklık derecesini verir.
- Program grafı (akış grafı) program akış diyagramından farklıdır: yalnızca düğümler ve kenarlar gösterilir.
- Her düğüm bir veya birden çok kaynak program deyimini temsil eder.
- Hesaplama yollarından biri: graftaki kapalı bölgeler ile grafın dışında kalan bölge birlikte sayılır.
- Slayttaki örnek (Pressman, Software Engineering, ortalama alma yöntemi): 13 düğümlü akış grafında R1 dış bölge, R2 ile R6 arası kapalı bölgelerdir. Toplam 6 bölge vardır, yani 6 temel yol bulunur.
- Aynı değer düğüm ve kenar sayısından da hesaplanabilir.
- Kullanım: en yüksek modül büyüklüğünü saptamak. Örneğin "döngüsel karmaşıklığı 10'dan büyük modül olmasın" kuralı konur.

**Büyüklük Ölçütleri (Yazılım Mühendisliği Tekrarı)**

- Doğrudan ölçüm: satır sayısı (LOC, lines of code) ve bin satır (KLOC). Satırlar doğrudan sayılır.
- Dolaylı ölçüm: işlev noktası (function point).
  - Tek bir değer değil, sayılan birkaç değere bağlı olarak bulunur.
  - Hocanın saydığı değerler: girdi sayısı, çıktı sayısı, dosya sayısı (veritabanındaki tablo sayısı), ekran yani ara birim sayısı.
  - Bu değerlerin ağırlıkları farklıdır. Örneğin dosya sayısı büyüklüğü daha çok etkiler.
  - Ham değer 14 soruyla düzeltilir. Sorular kalite özellikleriyle ilgilidir: dağıtık uygulama mı, çevrim içi veri girişi var mı, yeniden kullanılabilirlik önemli mi?
  - Her soru 0-5 arası puanlanır. Örneğin veritabanı ve uygulamasıyla tam dağıtık ise 5, yeniden kullanılabilirlik için az çaba gösterilmişse 1 verilir. Puanlar toplanıp düzeltme faktörü bulunur.

**Yazılım Güvenirliği**

- Güvenirlik (reliability) önemli bir kalite özelliğidir. Sonuç doğru (correctness) olabilir; soru bu doğruluğa ne kadar güvenileceğidir.
- Ticari bir yazılımda yaklaşık bin kod satırında bir hata çıkabilir; tamamen hatasız yazılım mümkün değildir.
- Güvenirlik geçmiş bilgilere ve teste dayalı başarı oranıdır:

$$\text{Güvenirlik} = \frac{\text{başarılı işletim süresi}}{\text{toplam işletim süresi}}$$

- Güvenirlik modelleri istatistikle çok ilişkilidir. Kullandıkları değerler: toplam hata sayısı, güvenirlik düzeyi, düzeltmeden sonra kalan hata sayısı.
- 1980'lerden beri 100'e yakın model geliştirilmiştir. En yaygın dört model (slayt):
  - Schneidewind modeli.
  - Jelinski-Moranda modeli.
  - Musa Basic ve Musa-Okumoto logaritmik Poisson yürütme zamanı (execution-time) modeli.
  - Littlewood-Verrall modeli.
- Model seçerken sorulanlar: Güvenirliği hesaplamak ne kadar kolay? Yazılımın tümüne mi, bir kısmına mı ihtiyaç var? Model kolay ve anlaşılır mı? Dış etkenlerden etkileniyor mu? Ne kadar test edilmeli?

**Yazılım Güvenirliğini Sağlama Adımları**

- Bell Laboratuvarları'nın çalışmasında, gelişimden teslime kadar 25 aktivite belirlenmiştir. Bazıları:
- Olabilirlik (fizibilite) ve gereksinimler:
  - İşlevsel profili belirlemek.
  - Hataları tanımlamak ve sınıflandırmak.
  - Müşteri güvenirlik ihtiyaçlarını tanımlamak.
  - Organizasyonu yönetmek.
- Tasarım ve uygulama:
  - Bileşenler arasında güvenirliği belirlemek.
  - İşlevsel kaynaklara odaklanmak.
  - Hatalı kodlamaları görmek.
  - Yazılım güvenirliğini ölçmek.
- Sistem testi: testleri yönetmek ve gözlemlemek, ek testleri belirlemek, test edilen nesnelerin güvenirliğini ölçmek.
- Teslim ve bakım: kullanıma geçişe doğru karar vermek, güvenirliği gözlemek ve kontrol altında tutmak, müşteri memnuniyetini izlemek.

**CK Metrikleri (Nesneye Yönelik Yazılımlar)**

- Chidamber ve Kemerer'in önerdiği ölçüm kümesidir. Nesneye yönelik yazılımlarda hangi özelliklere dikkat edilmesi gerektiğini gösterir.

| Metrik | Tanım | Yüksek olursa |
|---|---|---|
| WMC (Weighted Methods per Class), sınıfın ağırlıklı metot sayısı | Sınıftaki tüm metotların karmaşıklıklarının toplamı | Geliştirme ve bakım süresi artar; alt sınıflara etkisi büyür; sınıf uygulamaya özgü olur, yeniden kullanılabilirlik düşer |
| DIT (Depth of Inheritance Tree), kalıtım ağacının derinliği | Sınıfın kalıtım ağacının köküne uzaklığı; hiçbir sınıftan türetilmemişse 0 | Tasarım karmaşıklığı artar |
| NOC (Number of Children), alt sınıf sayısı | Sınıftan doğrudan türetilen alt sınıf sayısı | Kalıtımın yanlış kullanıldığını gösterebilir; metotlar daha çok test gerektirir, test bütçesi artar |
| CBO (Coupling Between Object Classes), nesne sınıfları arası bağımlılık | Sınıfın bağımlı olduğu sınıf sayısı (kalıtım dışı kullanım) | Modüler tasarıma zarar verir; yeniden kullanılabilirlik düşer; bakım zorlaşır; test maliyeti artar |

| Metrik | Tanım | Yüksek olursa |
|---|---|---|
| RFC (Response For a Class), sınıfın tetiklediği metot sayısı | Sınıftan bir nesnenin metotları çağrıldığında tetiklenebilecek tüm metotların sayısı | Test ve hata ayıklama zorlaşır; sınıf karmaşıktır |
| LCOM (Lack of Cohesion in Methods), metotlardaki uyum eksikliği | Metotların kullandığı nitelik kümelerinin kesişimlerinden oluşan ayrık kümelerin sayısı | Uyum düşüktür; LCOM'un düşük olması istenir |

- Bağımlılık tanımı: bir sınıfın özellik veya metotları başka bir sınıfta kullanılıyorsa ve aralarında kalıtım yoksa iki sınıf bağımlıdır.
- Yeniden kullanılabilir bir kütüphane parçası olmak için sınıfın başka sınıflara bağımlılığı az olmalı ve daha genel olmalıdır.

**Proje Gereksinim Belgesi (SRS)**

- IEEE 830 standardı SRS belgesinin hangi başlıklar ve içerikle hazırlanacağını gösterir. Standartların ürüne değil, süreç ve yönteme yönelik olduğuna da bir örnektir.
- IEEE 830 bölümleri:
  1. Giriş: amaç, belgenin hangi sürüme (release) ait olduğu, ürünün kapsamı.
  2. Genel tanım: işletim ortamı, ürün fonksiyonları, kullanıcı el kitabı, kabuller ve kısıtlar.
  3. Dış ara birim gereksinimleri: kullanıcı, donanım, yazılım ve iletişim ara birimleri.
  4. Sistem özellikleri: fonksiyonel gereksinimler (Requirement 1, Requirement 2, ...).
  5. Fonksiyonel olmayan gereksinimler: performans, güvenlik, kalite özellikleri, iş kuralları.
- Orijinal format çok ayrıntılıdır; aynısını kullanmak zorunlu değildir. İlk üç bölüm kısa tutulabilir.
- Gereksinimler baştan sona numaralanabilir ya da modüllere göre gruplanabilir. Hoca modül bazında yazmayı daha derli toplu bulur.
- Orta büyüklükte bir yazılımda toplam yaklaşık 50 gereksinim olur.
- Örnek öğrenci projesi (Web Accessible Alumni Database):
  - Fonksiyonel gereksinimler use case diyagramı ve her modül için use case belgeleriyle verilmiştir.
  - Ardından kullanıcı ara birim gereksinimleri ve performans gereksinimleri gibi fonksiyonel olmayan gereksinimler gelir.
- Teslim tarihi: öğrencilerin isteğiyle bir hafta ertelendi. SRS belgesi 17 Kasım 2020'de teslim edilecektir.

**Ders sonrası notlar**

- Hoca McCabe ölçüsü için "kapalı bölgeler artı en sondaki yarı açık bölge" dedi. Doğru ifade: kapalı bölgeler artı grafın dışında kalan bölge. Eşdeğer formüller: $V(G) = E - N + 2$ ($E$ kenar, $N$ düğüm sayısı) ve $V(G) = P + 1$ ($P$ karar düğümü sayısı).
- Hoca kara kutu testlerini sayarken "keşif testi, smoke test türü" dedi. Bu ikisi farklıdır. Keşif testi (exploratory testing) belge olmadan yazılımı kullanarak hata aramaktır. Duman testi (smoke test), yeni bir sürümün temel işlevlerinin çalışıp çalışmadığına bakan kısa bir ön testtir.
- WMC açıklanırken "sınıf sayısı çok olan sınıflar" dendi. Doğrusu "metot sayısı (ağırlıklı karmaşıklığı) çok olan sınıflar"dır.
- LCOM için hocanın verdiği tanım (ayrık nitelik kümelerinin sayısı) sonraki türev tanımlara yakındır. Chidamber ve Kemerer'in 1994 tanımında LCOM, ortak nitelik kullanmayan metot çiftlerinin sayısından ($P$) ortak nitelik kullanan çiftlerin sayısının ($Q$) çıkarılmasıdır: $P > Q$ ise $LCOM = P - Q$, değilse 0.
- İşlev noktası yönteminin standart beş öğesi: dış girdiler, dış çıktılar, dış sorgular, iç mantıksal dosyalar, dış ara birim dosyaları. Düzeltme faktörü $VAF = 0.65 + 0.01 \sum F_i$ ile, işlev noktası $FP = UFP \times VAF$ ile bulunur; $F_i$ 14 sorunun 0-5 arası puanlarıdır.
- McCall modelinin özgün üç grubu ürün işletimi (product operation), ürün gözden geçirme (product revision) ve ürün geçişidir (product transition). Hoca bunları kullanıcı, geliştirici ve işletici bakış açıları olarak anlattı.

- Bu derste hoca ölümcül ve büyük hatalarda teste genelde devam edilmediğini söyledi. Ders 4'teki slaytta ise kritik ve büyük hatalarda testin devam ettiği, ama yazılımın düzeltilmeden teslim edilmediği yazıyordu. Hoca bu sınırın kuruma göre değiştiğini de belirtti.
- Whisper altyazıda "haştat/hashtag metriği" Halstead, "makyap/Maccape" McCabe, "Musa Bezik" Musa Basic, "Lidlout" Littlewood, "Çitember Kemerer" Chidamber-Kemerer, "Prespa" Pressman olarak düzeltilmelidir.

**Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.